



?

Too low

Too high

Correct!

GROUP TWO

# A Guess Number Game

Expanded Challenge: a command-line number guessing game in Python

[Levels](#) · [Algorithm](#) · [Modules](#) · [game\(\)](#) · [Scoring](#) · [Leaderboard](#) · [Q&A](#)

---

# Contents

|                                             |    |
|---------------------------------------------|----|
| 1 Project Overview                          | 3  |
| 2 Introduction                              | 3  |
| 3 Project Progression: Level 1–4            | 5  |
| 4 General Algorithm of the Project          | 7  |
| 5 Python Modules Used                       | 7  |
| 6 Functions Created Before game()           | 9  |
| 7 The game() Function                       | 12 |
| 8 Detailed Code Walkthrough                 | 15 |
| 9 Technical Concepts Used                   | 19 |
| 10 Game State and Data Flow                 | 20 |
| 11 Single-Player and Multiplayer: One Logic | 22 |
| 12 Hint System                              | 22 |
| 13 Scoring and Rating System                | 23 |
| 14 Leaderboard System                       | 23 |
| 15 Level 1–3 Implementation Reference       | 24 |
| 16 What We Learned                          | 26 |
| 17 Important Questions and Answers          | 28 |
| 18 Conclusion                               | 29 |
| A Appendix: Complete Final Code             | 30 |
| B Appendix: Level 1–3 Demo Code             | 36 |

## 1

# Project Overview

| Item                 | Detail                                                                |
|----------------------|-----------------------------------------------------------------------|
| <b>Project title</b> | Number Guessing Game — Expanded Challenge                             |
| <b>Project type</b>  | Command-Line Interface (CLI) Python application                       |
| <b>Group</b>         | Group 2                                                               |
| <b>Final version</b> | Contained in the <code>game()</code> function; Level 4 of the project |

**Project goal.** The project is a number guessing game in which a computer generates a secret number and one or more players attempt to guess it. The game progressively introduces programming concepts, and the final implementation brings all of them together into one complete game system.

Concepts introduced along the way:

- random number generation
- functions
- loops
- conditional statements
- input validation
- exception handling
- difficulty levels
- attempt management
- hints
- scoring
- multiplayer turn management
- persistent data storage
- leaderboard management

## How to read this document

Sections 1–4 explain the idea and architecture. Sections 5–6 cover the modules and helper functions. Sections 7–8 explain `game()` in detail. Sections 9–14 cover state, scoring, hints and the leaderboard. Section 15 keeps the Level 1–3 code as reference, Section 16 lists what we learned, Section 17 collects questions and answers, Appendix A holds the complete final code, and Appendix B the Level 1–3 demonstration code.

## 2

# Introduction

A number guessing game is a simple programming problem in which the computer selects a number that the player does not know. The player repeatedly enters guesses, and the program compares each guess with the secret number and gives feedback:

| Feedback | Meaning                                               |
|----------|-------------------------------------------------------|
| Too high | The player's guess is greater than the secret number. |
| Too low  | The player's guess is smaller than the secret number. |
| Correct  | The player's guess matches the secret number.         |

Although the basic idea is simple, the project was designed to move from a one-time guess to a more complete game containing difficulty selection, limited attempts, input validation, hints, scoring, player ratings, multiplayer support, multiple rounds and leaderboard persistence.

The final version uses a **modular design**: different tasks are separated into different functions. The central function is `game()`. It coordinates the entire application and calls the other functions whenever their functionality is needed.

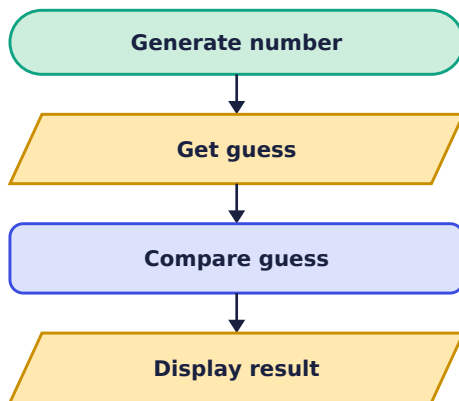
## 3

## Project Progression: Level 1-4

The project was designed in four levels representing the gradual development of the same basic idea.

### Level 1 — Basic

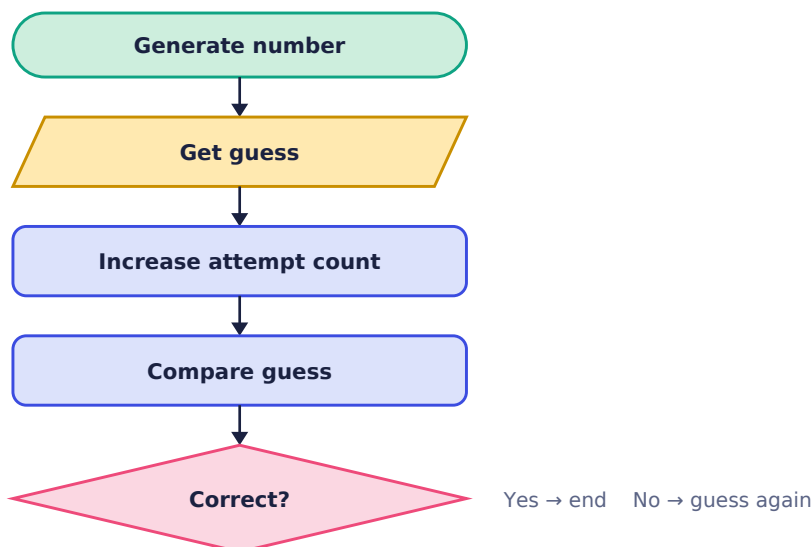
The first level introduces the fundamental concept. The program generates a random number, gets one guess from the player, compares the guess with the secret number and displays the result.



At this stage there is no repeated guessing, scoring, multiplayer system or leaderboard.

### Level 2 — Intermediate

Level 2 introduces repetition. Instead of allowing only one guess, the player can continue guessing until the correct number is found. A counter is also introduced to track attempts.



This level introduces `while` loops, counters, repeated input and termination conditions.

### Level 3 — Problem Solving

Level 3 makes the game more realistic. The player selects a difficulty, and the program validates input.

| Difficulty | Range | Attempts |
|------------|-------|----------|
| Easy       | 1-50  | 5        |
| Medium     | 1-100 | 7        |
| Hard       | 1-500 | 8        |

If the player enters `abc`, the program does not crash. If the player enters `700` while the range is 1-500, the program rejects the guess. Level 3 therefore introduces input validation, `try/except`, difficulty scaling, maximum attempts and game-over conditions.

## Level 4 — Expanded Challenge

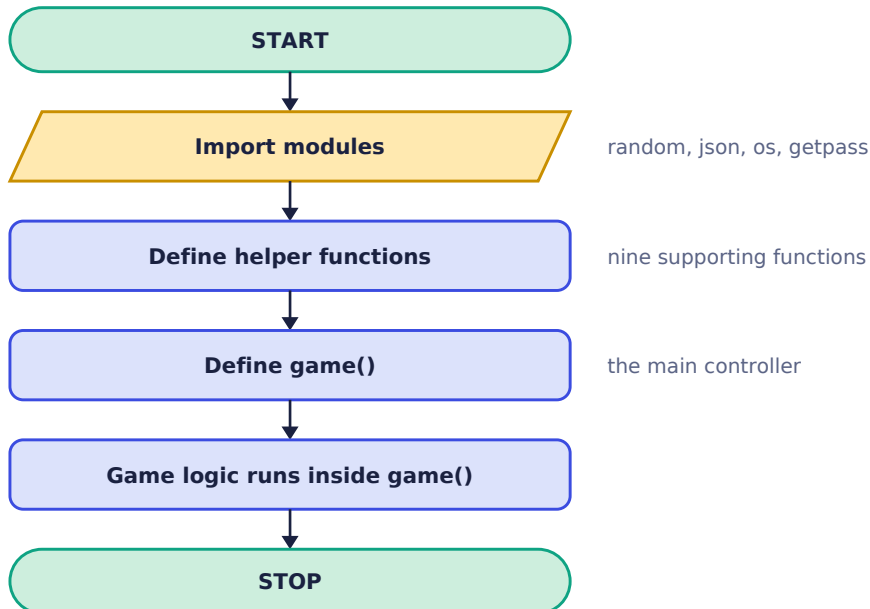
Level 4 combines the previous concepts and adds seven features:

| # | Feature            | Description                                                                     |
|---|--------------------|---------------------------------------------------------------------------------|
| 1 | Hint system        | After every three full rounds of wrong guesses, the player receives a hint.     |
| 2 | Scoring system     | The fewer attempts used, the higher the score.                                  |
| 3 | Rating system      | The score is converted into a star rating.                                      |
| 4 | Multiplayer mode   | Multiple players take turns guessing the same secret number.                    |
| 5 | Multiple rounds    | Players continue with new rounds without restarting the program.                |
| 6 | Leaderboard        | The program stores each player's best score.                                    |
| 7 | Persistent storage | The leaderboard lives in a JSON file and is loaded again when the program runs. |

## 4

## General Algorithm of the Project

This is the general algorithm of the entire program, not the detailed algorithm of `game()`. Its purpose is to show the overall architecture of the application.



### Helper functions defined before `game()`

|                                       |                                |                                    |
|---------------------------------------|--------------------------------|------------------------------------|
| <code>generate_secret_number()</code> | <code>compare_guess()</code>   | <code>choose_difficulty()</code>   |
| <code>get_valid_guess()</code>        | <code>calculate_score()</code> | <code>get_rating()</code>          |
| <code>choose_game_mode()</code>       | <code>give_hint()</code>       | <code>display_leaderboard()</code> |

#### Important distinction

The functions above `game()` do not independently run the entire game. They are helper functions that perform specific tasks `game()` needs. `game()` is the main controller: it coordinates those functions and determines the order in which the game operates.

Function definitions create reusable instructions; they do not execute those instructions immediately. The program only starts running the game when `game()` is called.

## 5

## Python Modules Used

The program imports four modules:

```
import random
import json
import os
import getpass
```

A **module** is a Python file or library containing code that provides functionality another program can reuse. Instead of implementing everything ourselves, Python lets us import existing functionality.

## 5.1 random

The `random` module generates pseudo-random values. The program uses `random.randint(low, high)`, which returns a random integer between `low` and `high`, **including both boundaries**.

```
random.randint(1, 100)  # could produce 37, 82, 1 or 100
```

**Why we use it:** the game needs a secret number the player cannot predict.

## 5.2 json

JSON stands for **JavaScript Object Notation**, a common text-based format for storing and exchanging structured data. The game uses it to store the leaderboard.

```
{
    "Mathias": 90,
    "John": 75
}
```

A Python dictionary can be converted into JSON and saved to a file. `json.dump()` writes Python data into JSON; `json.load()` reads JSON back into Python.

## 5.3 os

The `os` module lets Python interact with operating-system features. The program uses it for two purposes:

| Purpose                            | Code                                                                | Notes                                                                                 |
|------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Check whether a file exists</b> | <code>os.path.exists(leaderboard_file)</code>                       | Returns True or False.                                                                |
| <b>Clear the terminal</b>          | <code>os.system("cls")</code> or<br><code>os.system("clear")</code> | <code>cls</code> is generally for Windows; <code>clear</code> is for Linux and macOS. |

## 5.4 getpass

The `getpass` module reads input without displaying it normally on the terminal. The program uses `getpass.getpass()` in multiplayer mode, which helps prevent another player from immediately seeing the guess being typed.

## 6

## Functions Created Before game()

The program follows a **modular programming** approach. Instead of one enormous block of code, individual responsibilities are separated into functions, and each performs a particular job.

### 6.1 generate\_secret\_number()

```
def generate_secret_number(low=1, high=100):
    rand_num = random.randint(low, high)
    return rand_num
```

**Purpose:** generates the secret number. **Parameters:** `low=1` and `high=100` are *default parameters*. Calling `generate_secret_number()` uses 1 and 100; calling `generate_secret_number(1, 500)` uses 1 and 500. **Return value:** `rand_num` is returned to the caller.

```
secret_number = generate_secret_number(1, 50)
```

### 6.2 compare\_guess()

Compares `guess` with `secret_number`. It has three outcomes, which demonstrates **conditional branching**:

```
def compare_guess(guess, secret_number):
    if guess > secret_number:
        return "Too high! Try a lower number."
    elif guess < secret_number:
        return "Too low! Try a higher number."
    else:
        return "Correct"
```

The function *returns* a value instead of printing it. This separates the comparison logic from the presentation.

### 6.3 choose\_difficulty()

Lets the player select Easy, Medium or Hard inside a `while True` loop that repeats until the choice is 1, 2 or 3 (otherwise it prints "Invalid choice. Pick from [1, 2, 3]."). It returns three values, for example `return 1, 50, 5`, meaning `low = 1, high = 50, max_attempt = 5`. These are returned as a **tuple** and can be **unpacked**:

```
low, high, max_attempt = choose_difficulty()
```

### 6.4 get\_valid\_guess()

One of the most important validation functions: `def get_valid_guess(low, high, mode):` It uses `while True`: because it must keep asking until the input is valid.

- Single-player: `guess = input(...)`
- Multiplayer: `guess = getpass.getpass(...)`
- Input arrives as a string such as "50"; `guess = int(guess)` turns it into the integer 50.
- The check `if low <= guess <= high`: is a **chained comparison**, equivalent to `low <= guess` and `guess <= high`.
- If valid: `return guess`. Otherwise it prints "Out of range. Enter a number between low-high." and asks again. Invalid guesses do not count as attempts.

## 6.5 Exception handling

The conversion `int(guess)` can fail: `abc` cannot become an integer, so Python would raise `ValueError`. The program handles it so invalid input cannot crash the game:

```
try:
    guess = int(guess)

except ValueError:
    print("Invalid input. Enter a whole number.")
```

## 6.6 calculate\_score()

```
def calculate_score(attempts, max_attempt):
    score = ((max_attempt - attempts + 1) / max_attempt) * 100

    return round(score)
```

The fewer attempts the player uses, the higher the score. `round()` converts the decimal result into a rounded number.

## 6.7 get\_rating()

Converts a numerical score into a star rating, which separates the scoring calculation from the presentation:

```
def get_rating(score):
    if score <= 20:
        return "★"
    elif score <= 40:
        return "★★"
    elif score <= 60:
        return "★★★"
    elif score <= 80:
        return "★★★★"
    else:
        return "★★★★★"
```

| Score | 0-20 | 21-40 | 41-60 | 61-80 | 81-100 |
|-------|------|-------|-------|-------|--------|
| Stars | 1    | 2     | 3     | 4     | 5      |

The game prints star emoji; they are shown as ★ in this document.

## 6.8 choose\_game\_mode()

Asks the user to choose 1 (Single Player) or 2 (Multiplayer). The condition `if mode in ("1", "2"):` checks whether the entered value exists inside the tuple `("1", "2")`. Only valid choices are returned.

## 6.9 give\_hint()

The hint system uses two strategies. `wrong_guess` is the number of full wrong rounds counted by `game()`. When `wrong_guess == 3`, the program checks whether the secret number is even or odd using the **modulo operator** `%`, which returns the remainder of a division:

```
8 % 2 = 0    # even
9 % 2 = 1    # odd

if secret_number % 2 == 0:    # the number is even
```

Otherwise the function calculates a midpoint using **floor division** `//`:

```
midpoint = (low + high) // 2    # 10 // 3 = 3
```

The midpoint lets the program tell the player whether the secret number is in the lower or upper part of the range.

## 6.10 display\_leaderboard()

Displays the leaderboard, which is a dictionary such as `{"John": 70, "Mary": 90}`. The steps are:

| Step                    | Code                                  | Effect                               |
|-------------------------|---------------------------------------|--------------------------------------|
| <b>Convert to items</b> | <code>leaderboard.items()</code>      | Dictionary becomes sortable pairs.   |
| <b>Sort</b>             | <code>sorted(...)</code>              | Orders the pairs.                    |
| <b>Sort key</b>         | <code>key=lambda item: item[1]</code> | Sorts by the score (second element). |
| <b>Reverse</b>          | <code>reverse=True</code>             | Highest score first.                 |

If the leaderboard is empty it prints "No scores recorded yet." Otherwise each row is printed as `1. Mary - 90 points`, between a `===== LEADERBOARD =====` header and a closing line.

## 7

## The game() Function

**Purpose.** `game()` is the main controller of the final application. It does not perform every individual operation itself; it coordinates the helper functions.

### Overall structure

```
game()
├── Clear screen
├── Load/create leaderboard
├── Choose game mode
├── Determine number of players
├── Get player names
├── Repeat rounds
│   ├── Clear screen
│   ├── Choose difficulty
│   ├── Generate secret number
│   ├── Initialize game state
│   ├── Run guessing loop
│   ├── Determine winner
│   ├── Calculate score
│   ├── Update leaderboard
│   └── Show menu
```

## Pseudocode

```

START game
Clear the terminal
Set leaderboard file
IF leaderboard file does not exist
    Create an empty leaderboard and save it to the file
ELSE
    Load leaderboard from file
Choose game mode
IF single player: number of players = 1 ELSE: number of players = 2
Get each player's name; tidy it (strip, title); validate; store

WHILE playing
    Clear screen; display new round
    Choose difficulty; display range and attempts
    Generate secret number
    Create attempt counter for each player
    Initialize hint counters
    Set winner to none; first player is current player

    WHILE nobody has won AND current player has attempts
        Display current player's turn
        Get a valid guess
        Increase current player's attempt count
        Compare guess with secret number; display result and attempt count
        IF guess is correct
            Set current player as winner; display winner
            STOP guessing loop
        ELSE
            Record wrong guess
            IF all players have guessed incorrectly
                Increase full-round counter
                IF three full wrong rounds completed: display hint
            Move to next player

    Display round results
    IF there is a winner
        Calculate score and rating
        Update leaderboard; save leaderboard
    ELSE
        Display game-over message
    Display after-round menu
    IF play again: start another round
    ELSE IF leaderboard: display leaderboard
    ELSE IF exit: stop playing

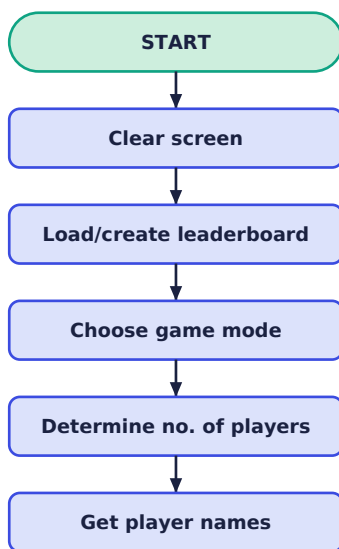
Display final leaderboard
Display thank-you message
END game

```

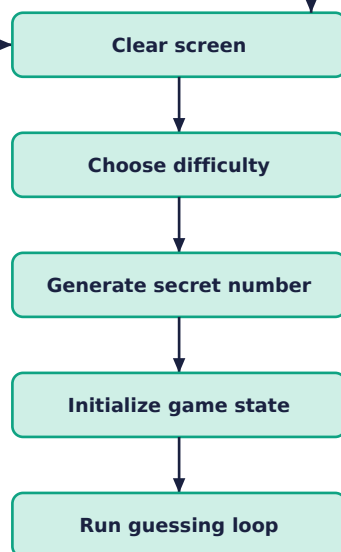
## Flowchart

Standard symbols: oval = start/end, rectangle = process, parallelogram = input/output, diamond = decision, arrows = flow.

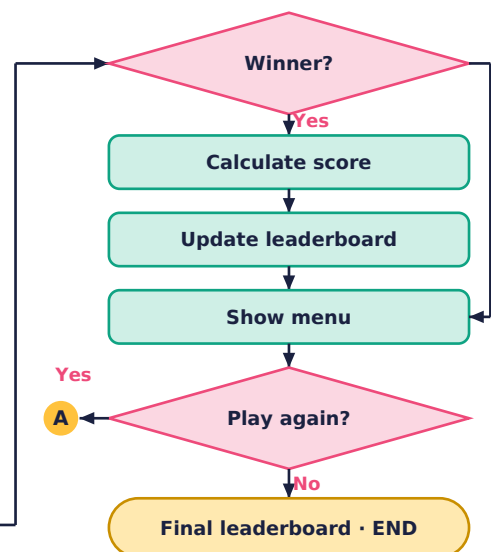
## SETUP · runs once



## EACH ROUND · repeats



## ROUND END



A = start the next round (back to Clear screen). Winner? No: game over, no score; the flow rejoins at Show menu.

## 8

## Detailed Code Walkthrough

### 8.1 Clearing the screen

```
if os.name == "nt":
    os.system("cls")
else:
    os.system("clear")
```

`os.name` identifies the operating-system family: `"nt"` on Windows and commonly `posix` on Unix-like systems. The program picks the appropriate command so the command-line interface stays clean, clearing the terminal before each new section or round.

### 8.2 The leaderboard file

```
leaderboard_file = "leaderboard.json"

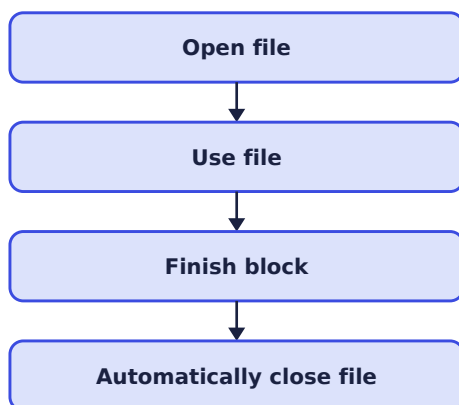
if not os.path.exists(leaderboard_file):
```

The first line stores the file name. `os.path.exists()` returns a **Boolean** (`True` or `False`) answering "does this file exist?". The `not` operator reverses the result, so the condition means: *if the file does NOT exist...*

### 8.3 with open() as file

```
with open(leaderboard_file, "w") as file:
```

This is a **context manager**. `open()` opens a file, and the mode `"w"` means *open for writing*. The file object is assigned to `file`. With `with`, Python automatically closes the file when the block finishes, which is safer and cleaner than manual closing.



### 8.4 JSON save and load

```
json.dump(
    leaderboard,
    file,
    indent=4
)

leaderboard = json.load(file)
```

`json.dump()` writes Python data into the file; `indent=4` keeps it readable. `json.load()` converts the JSON back into a Python object, which is why the leaderboard survives after the program closes.

## 8.5 Player management

```
player_names = []

for player_number in range(1, number_of_players + 1):
    ...
```

`player_names` is a **list**, an ordered collection. For one player `range(1, 2)` produces 1; for two players it produces 1, 2 (multiplayer in this project uses exactly two players). Each name is added with `player_names.append(name)`. The list preserves turn order, and an inner validation loop requires a non-empty, unique name:

```
while True:
    name = input(...).strip().title()
    if not name:
        print("Name cannot be empty.")
    elif name in player_names:
        print("That name is already taken. Choose another.")
    else:
        player_names.append(name)
        break
```

## 8.6 .strip() and .title()

`input(...).strip().title()` tidies the name in two steps. `.strip()` removes whitespace from both ends (" mathias " becomes "mathias"), and `.title()` capitalises each word ("mathias" becomes "Mathias"). Stray spaces stay out of names, and "john", "JOHN" and "John" all become the same name, so a player gets one leaderboard entry and the duplicate-name check catches them.

## 8.7 Round state

```
low, high, max_attempt = choose_difficulty()
secret_number = generate_secret_number(low, high)
attempts = [0] * number_of_players
wrong_guesses = 0
round_wrong_guesses = 0
winner = None
current_player = 0
```

**List multiplication:** with three players `[0] * 3` produces `[0, 0, 0]`, one counter per player by index.

| Player   | Index | Attempts so far |
|----------|-------|-----------------|
| Player 1 | 0     | 2               |
| Player 2 | 1     | 1               |
| Player 3 | 2     | 3               |

**winner = None.** `None` represents the absence of a value, so nobody has won yet. When a player wins, `winner = current_player` holds that player's index.

**current\_player = 0.** Python uses zero-based indexing: 0 is the first player, 1 the second, up to 4 for the fifth.

## 8.8 The main game loop

```
while (
    winner is None
    and attempts[current_player] < max_attempt
):
```

The loop continues only while two conditions are true: nobody has won, and the current player still has an attempt available.

## 8.9 Guess, compare, decide

```
if number_of_players > 1:
    print(f"\n{player_names[current_player]}'s turn")

guess = get_valid_guess(low, high, mode)
attempts[current_player] += 1
result = compare_guess(guess, secret_number)
print(result)
print(f"Attempt: {attempts[current_player]}/{max_attempt}")

if result == "Correct":
    winner = current_player
    print(f"\nCongratulations {player_names[winner]}!")
    break
```

Validation, comparison and overall state control are deliberately separate. Invalid guesses never consume an attempt, because `get_valid_guess()` only returns once the input is a valid integer in range.

## 8.10 Wrong guesses and hints

```
round_wrong_guesses += 1
if round_wrong_guesses == number_of_players:
    wrong_guesses += 1
    round_wrong_guesses = 0
    if wrong_guesses % 3 == 0:
        print("\n" + give_hint(secret_number, low, high, wrong_guesses))
```

Two counters separate an *individual* wrong guess from a *full cycle* in which every player guessed incorrectly. Hints are tied to full cycles (see Section 12). In single-player mode the loop then also prints **Attempts remaining: n**, while multiplayer keeps the output shorter.

## 8.11 Moving to the next player

```
current_player = (
    current_player + 1
) % number_of_players
```

Modulo wraps the index back to zero after the last player. With two players:  $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \dots$ ; with five:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$ .

## 8.12 Results, scoring and persistence

After the turn loop the program prints a round-results header and checks whether a winner exists. Only the **winner's** score and rating are calculated. The leaderboard then has three cases: a new name is added; a higher score replaces the saved best (printing the previous and new best); a lower or equal score changes nothing. The JSON file is written after every round that has a winner. If nobody won, it prints Game Over, the secret number and "No winner. No score was calculated."

```

if winner is not None:
    winner_name = player_names[winner]
    winner_attempts = attempts[winner]
    score = calculate_score(winner_attempts, max_attempt)
    rating = get_rating(score)

    if winner_name not in leaderboard:
        leaderboard[winner_name] = score
    elif score > leaderboard[winner_name]:
        leaderboard[winner_name] = score

    with open(leaderboard_file, "w") as file:
        json.dump(leaderboard, file, indent=4)
else:
    print("Game Over!")
    print(f"The secret number was {secret_number}.")
    print("\nNo winner. No score was calculated.")

```

## 8.13 After-round menu

| Option                      | Action                                                                                           |
|-----------------------------|--------------------------------------------------------------------------------------------------|
| <b>1 — Play again</b>       | Breaks the menu loop and begins another round.                                                   |
| <b>2 — View leaderboard</b> | Displays the leaderboard, then returns to the menu.                                              |
| <b>3 — Exit</b>             | Sets <code>playing = False</code> , ending the outer round loop; the final leaderboard is shown. |

When the round loop ends, the final leaderboard is displayed, followed by "Thanks for playing!".

**Starting the program.** The last lines of the file are `if __name__ == "__main__":` followed by `game()`. The game starts when the file is run directly, but not when the file is imported by another program.

## 9

# Technical Concepts Used

| Concept                                   | Where and how it appears                                                                                                                                                        |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Functions, parameters, arguments</b>   | Helpers receive values (low, high, mode, score); callers pass arguments such as <code>generate_secret_number(1, 500)</code> .                                                   |
| <b>Return values</b>                      | Helpers send results back, e.g. <code>compare_guess()</code> returns text instead of printing it.                                                                               |
| <b>Local variables / global constants</b> | Variables like <code>secret_number</code> and <code>leaderboard_file</code> are local to their functions; the single <code>game()</code> call at the bottom starts the program. |
| <b>Lists</b>                              | <code>player_names</code> and <code>attempts</code> hold ordered collections.                                                                                                   |
| <b>Dictionaries</b>                       | <code>leaderboard</code> maps player names to best scores.                                                                                                                      |
| <b>Tuples and unpacking</b>               | <code>choose_difficulty()</code> returns three values assigned to three variables; <code>("1", "2")</code> is a tuple used for membership checks.                               |
| <b>Indexing</b>                           | <code>player_names[winner]</code> retrieves the winning name; indexes start at 0.                                                                                               |
| <b>Loops, while True, break</b>           | Round loop, turn loop and validation loops; <code>break</code> exits a loop early.                                                                                              |
| <b>Conditional branching</b>              | <code>if / elif / else</code> in comparison, menus, validation and scoring.                                                                                                     |
| <b>Exception handling</b>                 | <code>try/except ValueError</code> around integer conversion.                                                                                                                   |
| <b>File handling and JSON</b>             | <code>open()</code> , read/write modes, <code>with</code> , <code>json.dump()</code> and <code>json.load()</code> .                                                             |
| <b>Booleans and None</b>                  | Conditions evaluate to True/False; <code>winner = None</code> means no winner yet.                                                                                              |
| <b>os features</b>                        | <code>os.name</code> , <code>os.path.exists()</code> , <code>os.system()</code> .                                                                                               |
| <b>Strings and f-strings</b>              | <code>f"Attempt: {attempt}"</code> embeds values in text; <code>.strip()</code> cleans input.                                                                                   |
| <b>List multiplication</b>                | <code>[0] * number_of_players</code> builds one counter per player.                                                                                                             |
| <b>Modulo and floor division</b>          | <code>%</code> for even/odd, cyclic turns and every third wrong cycle; <code>//</code> for the midpoint.                                                                        |
| <b>Lambda, sorted(), enumerate()</b>      | Lambda is the sort key; <code>sorted()</code> orders entries; <code>enumerate()</code> adds ranking positions.                                                                  |
| <b>Functional-style elements</b>          | Small returning functions and lambda-based sorting, although the whole program is not purely functional.                                                                        |
| <b>State management</b>                   | Attempts, turn, winner and playing variables describe the current state of the game.                                                                                            |
| <b>__name__ guard</b>                     | <code>if __name__ == "__main__":</code> runs <code>game()</code> only when the file is run directly, not when it is imported.                                                   |

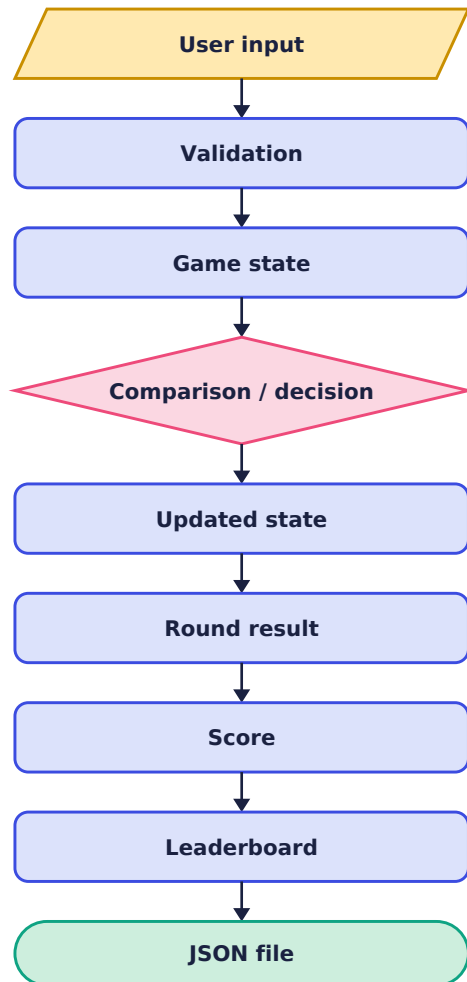
## 10

## Game State and Data Flow

The Level 4 program is **state-driven**. State is information that changes as the game progresses.

| Variable                         | Meaning                            | How it changes                              |
|----------------------------------|------------------------------------|---------------------------------------------|
| <code>playing</code>             | Whether another round should run   | False when the user exits                   |
| <code>number_of_players</code>   | Players in the current mode        | Set before rounds                           |
| <code>player_names</code>        | Names in turn order                | Built before rounds                         |
| <code>low / high</code>          | Current number range               | Changed by difficulty                       |
| <code>max_attempt</code>         | Current attempt limit              | Changed by difficulty                       |
| <code>secret_number</code>       | Hidden target                      | Generated each round                        |
| <code>attempts</code>            | Attempts used by each player       | Incremented after valid guesses             |
| <code>wrong_guesses</code>       | Completed full wrong cycles        | Incremented after every player's wrong turn |
| <code>round_wrong_guesses</code> | Wrong guesses in the current cycle | Incremented, then reset                     |
| <code>winner</code>              | Winner's index, or None            | Set after a correct guess                   |
| <code>current_player</code>      | Current turn index                 | Moves cyclically                            |
| <code>leaderboard</code>         | Best saved score per player        | Loaded, updated and saved                   |

## Data flow



## 11

## Single-Player and Multiplayer: One Logic

The same turn loop handles both modes. Instead of writing separate code, the **number of players** controls how many turns happen per cycle.

| Aspect                    | Single-player                               | Multiplayer                                                   |
|---------------------------|---------------------------------------------|---------------------------------------------------------------|
| <b>Players</b>            | <code>number_of_players = 1</code>          | More than one (the current implementation sets 2)             |
| <b>Guess input</b>        | <code>input()</code>                        | <code>getpass.getpass()</code> so others cannot see the guess |
| <b>Attempts</b>           | A list of one counter                       | One counter per player                                        |
| <b>Next player</b>        | $(0 + 1) \% 1 = 0$ , always the same player | Rotates $0 \rightarrow 1 \rightarrow 0 \dots$                 |
| <b>Wrong-cycle</b>        | Every wrong guess completes a cycle         | Completed when all players have missed                        |
| <b>Turn label</b>         | Not shown                                   | Prints the player's name before each turn                     |
| <b>Intro text</b>         | "You have N attempts."                      | "Each player has N attempts."                                 |
| <b>Attempts remaining</b> | Printed after each wrong guess              | Not printed, to keep output short                             |

Because turn rotation uses modulo, the same expression  $(\text{current\_player} + 1) \% \text{number\_of\_players}$  works for any number of players. This reduces duplication and makes the design scalable.

## 12

## Hint System

Hints appear after every three *full* rounds of wrong guesses. A full round means every player has guessed incorrectly once.

| Turn | Player   | Result       | round_wrong_guesses      | wrong_guesses   |
|------|----------|--------------|--------------------------|-----------------|
| 1    | Player 1 | Wrong        | 1                        | 0               |
| 2    | Player 2 | Wrong        | 2 = players → reset to 0 | 1               |
| 3-4  | P1, P2   | Wrong, Wrong | cycle complete           | 2               |
| 5-6  | P1, P2   | Wrong, Wrong | cycle complete           | 3 → <b>hint</b> |

**Which hint?** `game()` requests a hint whenever `wrong_guesses \% 3 == 0`. Inside `give_hint()`, the first hint (`wrong_guess == 3`) reports *even* or *odd*; later hints use the midpoint to narrow the range.

```
# Range 1-100, midpoint = (1 + 100) // 2 = 50
# secret_number <= 50 -> "between 1 and 50"
# secret_number > 50 -> "between 51 and 100"
```

## 13

## Scoring and Rating System

The score rewards fewer attempts:  $\text{score} = ((\text{max\_attempt} - \text{attempts} + 1) / \text{max\_attempt}) * 100$ , rounded. A first-attempt win is always 100.

| Attempts used | Easy (max 5)  | Medium (max 7) |
|---------------|---------------|----------------|
| 1             | 100 (5 stars) | 100 (5 stars)  |
| 2             | 80 (4)        | 86 (5)         |
| 3             | 60 (3)        | 71 (4)         |
| 4             | 40 (2)        | 57 (3)         |
| 5             | 20 (1)        | 43 (3)         |
| 6             | -             | 29 (2)         |
| 7             | -             | 14 (1)         |

The rating converts the score into stars: 20 or less gives one star, 21–40 two, 41–60 three, 61–80 four, and above 80 five. Example: a Medium win on attempt 6 scores 29, which is two stars.

## 14

## Leaderboard System

- **Stored** in `leaderboard.json` as a dictionary mapping each name to their **best** score.
- **Created** as an empty dictionary the first time the program runs; **loaded** on later runs.
- **Updated** only when a winner's new score beats their saved score. A lower new score never replaces a higher one.
- **Saved** after every round that has a winner.
- **Displayed** sorted highest-first with ranking numbers from `enumerate()`; an empty leaderboard prints "No scores recorded yet."

```
{
    "Mary": 90,
    "John": 70
}
```

```
# Displayed as:
# ===== LEADERBOARD =====
# 1. Mary - 90 points
# 2. John - 70 points
# =====
```

## 15

## Level 1-3 Implementation Reference

These earlier versions are kept as reference. They show how each level adds one idea and reuses the helper functions.

### Not in the final file

The final code contains only the helper functions and `game()`; these level functions were earlier stages. The demonstration file that contains them is in Appendix B. In the final code `get_valid_guess()` also takes a `mode` argument.

### Level 1

```
def level1():
    secret_number = generate_secret_number()
    guess = int(input("Guess a number between 1 and 100: "))
    result = compare_guess(guess, secret_number)
    print(result)
```

One complete cycle: generate → input → compare → display. No repeated guessing or attempt limit.

### Level 2

```
def level2():
    secret_number = generate_secret_number()
    attempt = 0
    while True:
        guess = int(input("Guess a number between 1 and 100: "))
        attempt += 1
        result = compare_guess(guess, secret_number)
        print(f"{result}. Attempt: {attempt}")
        if guess == secret_number:
            print(f"You win. Total attempts: {attempt}")
            break
```

Introduces repetition with `while True`, an attempt counter, and `break` to stop on the correct guess.

### Level 3

```
def level3():
    low, high, max_attempt = choose_difficulty()
    print(f"You have {max_attempt} attempts.")
    secret_number = generate_secret_number(low, high)
    attempt = 0
    while attempt < max_attempt:
        guess = get_valid_guess(low, high)
        attempt += 1
        result = compare_guess(guess, secret_number)
        print(result)
        if result == "Correct":
            print(f"Congratulations! You got it in {attempt} attempts.")
            break
        if attempt == max_attempt:
            print(f"Game Over! Secret number was: {secret_number}")
        else:
            print(f"You have {max_attempt - attempt} attempts left.")
```

Adds difficulty selection, strict validation, a maximum attempt count and game-over handling.

| Level | Main change                                                                  |
|-------|------------------------------------------------------------------------------|
| 1     | One guess + comparison.                                                      |
| 2     | Repeated guesses + attempt counter.                                          |
| 3     | Difficulty + validation + attempt limit.                                     |
| 4     | Hints + scoring + rating + multiplayer + rounds + leaderboard + persistence. |

## 16

## What We Learned

The main takeaways from building the project, each with a short definition and where it appears in our game.

### Python tools

| Topic                | What it is                                                                                                             | In our game                                                            |
|----------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>random module</b> | A built-in module that generates random values.                                                                        | <code>random.randint(low, high)</code> picks the secret number.        |
| <b>JSON module</b>   | JSON is a text format for structured data; the <code>json</code> module saves Python data to a file and reads it back. | <code>leaderboard.json</code> keeps best scores after the game closes. |
| <b>getpass</b>       | Reads typed input without showing it on the screen.                                                                    | Hides guesses in multiplayer so the other player cannot see them.      |
| <b>Unpacking</b>     | Assigning several returned values to several variables at once.                                                        | <code>low, high, max_attempt = choose_difficulty()</code>              |

### Control flow and safety

| Topic                                   | What it is                                                                                       | In our game                                                                                                    |
|-----------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Functional programming</b>           | Building a program from small functions that each do one job and return a result.                | <code>compare_guess()</code> , <code>calculate_score()</code> and <code>get_rating()</code> each do one thing. |
| <b>While loop control flow</b>          | A <code>while</code> loop repeats while a condition is true; <code>break</code> leaves it early. | Keeps asking for guesses until a win or the attempt limit.                                                     |
| <b>Conditional branching</b>            | <code>if / elif / else</code> lets the program choose between paths.                             | <code>compare_guess()</code> returns Too high, Too low or Correct.                                             |
| <b>Input validation with try/except</b> | Checks input and catches errors instead of crashing.                                             | <code>"abc"</code> raises <code>ValueError</code> and is asked again; numbers out of range are rejected.       |

## Game logic

| Topic                                          | What it is                                                      | In our game                                                                                |
|------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Attempt counter management</b>              | Tracking how many guesses each player has used against a limit. | <code>attempts = [0] * number_of_players</code> keeps one counter per player.              |
| <b>Difficulty scaling logic</b>                | Changing the rules to make the game easier or harder.           | Easy 1-50 with 5 attempts, Medium 1-100 with 7, Hard 1-500 with 8.                         |
| <b>Scoring algorithms</b>                      | A formula that turns performance into a score.                  | <code>((max_attempt - attempts + 1) / max_attempt) * 100</code> , then converted to stars. |
| <b>Multiplayer turn-based state management</b> | Tracking whose turn it is and each player's progress.           | <code>(current_player + 1) % number_of_players</code> rotates the turn.                    |

## 17

## Important Questions and Answers

---

**Q1. Why divide the program into functions?**

To make it modular, reusable and easier to understand, test and debug. Each function has one responsibility.

**Q2. Why does `compare_guess()` return text instead of printing?**

Returning a value separates comparison logic from presentation, so the caller decides what to do with the result.

**Q3. Why use `while True` in `get_valid_guess()`?**

We don't know how many attempts the user needs to enter valid input, so the loop continues until a valid guess is returned.

**Q4. Why use `try/except`?**

Converting text such as 'abc' with `int()` raises `ValueError`. Catching it prevents a crash and lets the program re-ask.

**Q5. Why are modes compared with "1" and "2" but guesses converted with `int()`?**

`input()` always returns a string. Menu choices can be compared as strings; guesses need to be integers for numeric comparison.

**Q6. Do invalid guesses cost an attempt?**

No. `get_valid_guess()` only returns after the input is a valid integer inside the selected range.

**Q7. Why use `with open()`?**

It closes the file automatically when the block ends, even if something goes wrong, which is safer than manual closing.

**Q8. Why is the leaderboard stored in JSON?**

JSON is a simple text format that maps naturally to Python dictionaries and persists data between runs.

**Q9. How does turn rotation work for any number of players?**

`current_player = (current_player + 1) % number_of_players` wraps the index back to zero after the last player.

**Q10. Does the game support more than two players?**

The turn logic can: turns rotate with modulo and every counter is a list. In this project multiplayer sets exactly two players. Asking for the player count would extend it.

**Q11. When does a hint appear and which type?**

After every third full wrong cycle. The first hint is even/odd; later ones narrow the range with the midpoint.

**Q12. Does a lower score replace the saved best score?**

No. The leaderboard stores each player's best score only.

**Q13. Why is `.title()` used on player names?**

So "john", "JOHN" and "John" all become "John". A player then has one leaderboard entry, and the duplicate-name check in multiplayer treats them as the same name.

**Q14. Is the leaderboard saved when the winner does not beat their old score?**

The file is written after every round that has a winner, but a name's stored score only changes if it is new or the new score is higher.

**Q15. Does defining `game()` run it?**

No. Defining a function only creates it. The check at the bottom, if `__name__ == "__main__":` `game()`, starts the application when the file is run directly.

**18**

## Conclusion

The Number Guessing Game shows how a simple Python exercise becomes a structured command-line application. It progresses from random number generation and comparison to loops, validation, difficulty levels, hints, scoring, multiplayer turns, multiple rounds and persistent storage.

The central `game()` function coordinates the application while helper functions handle focused tasks. This separation makes the code easier to explain, test, debug and extend.

**Key lesson**

Complex behaviour can be built from smaller, understandable components. The main data flow is: user input → validation → game state → comparison → decision → score → leaderboard → persistent storage.

## A

## Appendix: Complete Final Code

The complete final program. Star emoji are shown as ★ and a few long comment lines are wrapped to fit the page; otherwise it is unchanged.

```
import random
import json
import os
import getpass

def generate_secret_number(low=1, high=100):
    """Returns a random secret number."""
    rand_num = random.randint(low, high)
    return rand_num

def compare_guess(guess, secret_number):
    """Compares the player's guess with the secret number and returns
    'Too high', 'Too low', or 'Correct'."""
    if guess > secret_number:
        return "Too high! Try a lower number."
    elif guess < secret_number:
        return "Too low! Try a higher number."
    else:
        return "Correct"

def choose_difficulty():
    """Returns the low range, high range and maximum attempts based on the
    selected difficulty."""
    print("\nDifficulty options")
    print("1. Easy")
    print("2. Medium")
    print("3. Hard")

    while True:
        choice = input("Choose difficulty level: ")

        if choice == "1":
            return 1, 50, 5

        elif choice == "2":
            return 1, 100, 7

        elif choice == "3":
            return 1, 500, 8

        else:
            print("Invalid choice. Pick from [1, 2, 3].")

def get_valid_guess(low, high, mode):
    """
    Returns a valid integer guess within the given range.
    Invalid guesses do not count as attempts.

    In single-player mode, input() is used so the guess
    is visible.

    In multiplayer mode, getpass.getpass() is used so
    the guess is hidden from the other players.
    """
    while True:
        # Single-player mode
        if mode == "1":
            guess = input(f"Guess a number ({low}-{high}): ")
            if guess.isdigit():
                guess = int(guess)
                if guess >= low and guess <= high:
                    return guess
            else:
                print("Invalid guess. Please enter a number.")
        else:
            guess = getpass.getpass(f"Guess a number ({low}-{high}): ")
            if guess.isdigit():
                guess = int(guess)
                if guess >= low and guess <= high:
                    return guess
            else:
                print("Invalid guess. Please enter a number.")
```

```

# Multiplayer mode
else:
    guess = getpass.getpass(f"Guess a number ({low}-{high}): ")

    try:
        guess = int(guess)

        if low <= guess <= high:
            return guess

        else:
            print(f"Out of range. Enter a number between {low}-{high}.")

    except ValueError:
        print("Invalid input. Enter a whole number.")

def calculate_score(attempts, max_attempt):
    """ Calculates a player's score. Fewer attempts result in a higher score."""
    score = ((max_attempt - attempts + 1) / max_attempt) * 100

    return round(score)

def get_rating(score):
    """ Returns a star rating based on the player's score."""
    if score <= 20:
        return "*"
    elif score <= 40:
        return "**"
    elif score <= 60:
        return "***"
    elif score <= 80:
        return "****"
    else:
        return "*****"

def choose_game_mode():
    """Ask the player to choose a game mode and return the valid choice."""
    while True:
        print("\nChoose game mode:")
        print("1. Single Player")
        print("2. Multiplayer")

        mode = input("Enter your choice: ")

        if mode in ("1", "2"):
            return mode

        print("Invalid game mode. Please choose 1 or 2.")

def give_hint(secret_number, low, high, wrong_guess):
    """Generate and return a hint about the secret number."""
    if wrong_guess == 3:

        if secret_number % 2 == 0:
            return "Hint: The number is even."

        else:
            return "Hint: The number is odd."

    else:

        midpoint = (low + high) // 2

        if secret_number <= midpoint:
            return f"Hint: The number is between {low} and {midpoint}."

        else:
            return (f"Hint: The number is between {midpoint + 1} and {high}.")

def display_leaderboard(leaderboard):

```

```

"""
Displays the current leaderboard.
"""
print("\n===== LEADERBOARD =====")

if not leaderboard:
    print("No scores recorded yet.")

else:
    sorted_leaderboard = sorted(leaderboard.items(),
                                key=lambda item: item[1], reverse=True)

    for position, (name, score) in enumerate(sorted_leaderboard, start=1):
        print(f"{position}. {name} - {score} points")

print("=====")

def game()::
    """
    Runs the full challenge version of the number guessing game.

    Single player and multiplayer share ONE game loop.
    Single player is simply a game with 1 player, so the
    same code handles multiple players without duplication.
    """

    # CLEAR SCREEN
    if os.name == "nt":
        os.system("cls")
    else:
        os.system("clear")

    leaderboard_file = "leaderboard.json"

    # LOAD OR CREATE LEADERBOARD
    if not os.path.exists(leaderboard_file):

        leaderboard = {}

        with open(leaderboard_file, "w") as file:
            json.dump(leaderboard, file, indent=4)

    else:

        with open(leaderboard_file, "r") as file:
            leaderboard = json.load(file)

    # CHOOSE GAME MODE
    mode = choose_game_mode()

    # DECIDE NUMBER OF PLAYERS
    # Single player -> exactly 1 player.
    # Multiplayer -> exactly 2 players for the scope of this project.
    if mode == "1":
        number_of_players = 1

    else:
        number_of_players = 2

    # GET PLAYER NAMES
    player_names = []

    # Loop once per player (runs only once in single player)
    for player_number in range(1, number_of_players + 1):

        while True:

            name = input(f"Enter Player {player_number} name: ").strip().title()

            # Name must not be empty
            if not name:
                print("Name cannot be empty.")

            # Names must be unique, otherwise two players would share

```

```

        # one leaderboard entry
        elif name in player_names:
            print("That name is already taken. Choose another.")

        else:
            player_names.append(name)
            break

# PLAY MULTIPLE ROUNDS
playing = True

while playing:

    # CLEAR SCREEN FOR NEW ROUND
    if os.name == "nt":
        os.system("cls")
    else:
        os.system("clear")

    print("\n=====")
    print("                NEW ROUND")
    print("=====")

    # CHOOSE DIFFICULTY
    low, high, max_attempt = choose_difficulty()

    # Single player sees "You have ...", multiplayer
    # sees "Each player has ..."
    if number_of_players == 1:
        print(f"\nYou have {max_attempt} attempts.")
    else:
        print(f"\nEach player has {max_attempt} attempts.")

    print(f"The number is between {low} and {high}.")

    # GENERATE SECRET NUMBER
    secret_number = generate_secret_number(low, high)

    # INITIALIZE GAME STATE
    # Each player has their own attempt counter.
    # Example with 2 players: [0, 0]
    attempts = [0] * number_of_players

    # Counts how many FULL rounds of wrong guesses have happened
    # (a full round = every player guessed once and all were wrong).
    # Used to trigger hints.
    wrong_guesses = 0

    # Counts wrong guesses inside the CURRENT full round. When it reaches
    # number_of_players, one full round of wrong guesses is complete,
    # then it resets to 0. (In single player, every wrong guess is a
    # full round.)
    round_wrong_guesses = 0

    # Stores the index of the winning player
    winner = None

    # Player 0 starts
    current_player = 0

    # MAIN GAME LOOP (it loops based on the number of players you have)
    # The loop stops when someone wins, or when the current player has
    # used all attempts. Everyone takes turns in the same order, so when
    # the player at the start of the order is out of attempts, every
    # player is out.
    while (winner is None and attempts[current_player] < max_attempt):

        # Only show whose turn it is when more than one player is playing
        if number_of_players > 1:
            print(f"\n{player_names[current_player]}'s turn")

        guess = get_valid_guess(low, high, mode)

```

```

    attempts[current_player] += 1

    result = compare_guess(guess, secret_number)

    print(result)

    print(f"Attempt: {attempts[current_player]}/{max_attempt}")

    # PLAYER WINS
    if result == "Correct":

        winner = current_player

        print(f"\nCongratulations {player_names[winner]}! "
              f"The secret number is {secret_number}.")

        break

    # PLAYER GUESSED WRONG
    round_wrong_guesses += 1

    # Once every player has guessed wrong, one full round of wrong
    # guesses is complete.
    if round_wrong_guesses == number_of_players:

        wrong_guesses += 1

        round_wrong_guesses = 0

        # Give a hint after every 3 full rounds of wrong guesses
        if wrong_guesses % 3 == 0:

            print("\n" + give_hint(secret_number, low, high,
                                   wrong_guesses))

    # Show remaining attempts for the player who just guessed
    # (single player only, to keep multiplayer output short)
    if (number_of_players == 1 and
        attempts[current_player] < max_attempt):

        print(f"Attempts remaining: "
              f"{max_attempt - attempts[current_player]}\n")

    # Move to the next player. The % operator wraps back to player 0
    # after the last player. With 1 player this stays on player 0.
    current_player = (current_player + 1) % number_of_players

# ROUND RESULTS
print("\n===== ROUND RESULTS =====")

if winner is not None:

    print(f"Winner: {player_names[winner]}")

    # CALCULATE WINNER'S SCORE ONLY
    winner_name = player_names[winner]

    winner_attempts = attempts[winner]

    score = calculate_score(winner_attempts, max_attempt)

    rating = get_rating(score)

    print(f"\n{winner_name}")

    print(f"Attempts: {winner_attempts}")

    print(f"Score: {score} points")

    print(f"Rating: {rating}")

    # UPDATE LEADERBOARD
    if winner_name not in leaderboard:

```

```

        leaderboard[winner_name] = score

        print(f"\n{winner_name} has been added to the leaderboard "
              f"with {score} points.")

    elif score > leaderboard[winner_name]:

        old_score = leaderboard[winner_name]

        leaderboard[winner_name] = score

        print(f"\nNew best score for {winner_name}!")

        print(f"Previous best: {old_score}")

        print(f"New best: {score}")

    # SAVE LEADERBOARD
    with open(leaderboard_file, "w") as file:

        json.dump(leaderboard, file, indent=4)

else:
    # NO WINNER
    print("Game Over!")

    print(f"The secret number was {secret_number}.")

    print("\nNo winner. No score was calculated.")

# AFTER-ROUND MENU
while True:

    print("\n===== MENU =====")
    print("1. Play another round")
    print("2. View leaderboard")
    print("3. Exit")

    choice = input("Choose an option: ")

    if choice == "1":

        break

    elif choice == "2":

        display_leaderboard(leaderboard)

    elif choice == "3":

        playing = False

        break

    else:

        print("Invalid choice. Pick 1, 2 or 3.")

# FINAL LEADERBOARD
print("\n")

display_leaderboard(leaderboard)

print("\nThanks for playing!")

if __name__ == "__main__":
    game()

```

## B

## Appendix: Level 1-3 Demo Code

The demonstration file used to show levels 1 to 3, exactly as written. It ends by calling `level3()`; the `level2()` and `level1()` calls are commented out and can be switched on to demonstrate the earlier levels.

### Differences from the final game()

In this file Hard allows **10** attempts (the final game() uses 8), `get_valid_guess()` takes no `mode` argument, and `choose_difficulty()` converts the choice with `int(input())`, so text such as `abc` would raise a `ValueError` there. The final version reads the choice as text instead.

```
#Group 2
# Guess number game
import random

import json
import os

def generate_secret_number(low=1, high=100):
    """
    returns a random secret number
    """
    rand_num = random.randint(low, high)
    return rand_num

def compare_guess(guess, secret_number):
    """
    Compares the player's guess with the secret number and return any of 'too high', 'too low' or 'correct'
    """
    if guess > secret_number:
        return "Too high! try a lower number"
    elif guess < secret_number:
        return "Too low! try a higher number"
    else:
        return "Correct"

def choose_difficulty():
    """
    returns low, high and maximum attempt
    """
    print("Difficulty options\n1. Easy\n2. Medium\n3. Hard")
    while True:
        choice = int(input("Choose difficulty level: "))
        if choice == 1:
            return 1, 50, 5
        elif choice == 2:
            return 1, 100, 7
        elif choice == 3:
            return 1, 500, 10
        else:
            print("Invalid choice. Pick from [1. Easy, 2. Medium, 3. Hard]")
            continue

def get_valid_guess(low, high):
    """
    It returns a valid guess and attempt
    """
    while True:
        guess = input(f"Guess a number between {low}-{high}: ")
        try:
            guess = int(guess)
            # guess >= low and guess <= high
            # low <= guess and guess <= high
            if low <= guess <= high:
                return guess
```

```
        else:
            print(f"Out of range. Enter between {low}-{high}")
        except ValueError:
            print("Invalid input.")

def level1():
    """
    runs the simplest version of the number guessing game.
    """
    secret_number = generate_secret_number()
    guess = int(input("Guess a number between 1 and 100: "))
    result = compare_guess(guess, secret_number)
    print(result)

def level2():
    """
    Allow the player to keep guessing
    """
    secret_number = generate_secret_number()
    attempt = 0
    while True:
        guess = int(input("Guess a number between 1 and 100: "))
        attempt += 1
        result = compare_guess(guess, secret_number)
        print(f"{result}. Attempt: {attempt}")
        if guess == secret_number:
            print(f"You win. Total attempts: {attempt}")
            break

def level3():
    low, high, max_attempt = choose_difficulty()

    print(f"You have {max_attempt} attempts.")

    secret_number = generate_secret_number(low, high)
    attempt = 0

    while attempt < max_attempt:
        guess = get_valid_guess(low, high)
        attempt += 1

        result = compare_guess(guess, secret_number)
        print(result)

        if result == "Correct":
            print(f"Congratulations! You got it in {attempt} attempts.")
            break

        if attempt == max_attempt:
            print(f"Game Over! Secret number was: {secret_number}")
        else:
            print(f"You have {max_attempt - attempt} attempts left.")

level3()
#level2()
#level1()
```